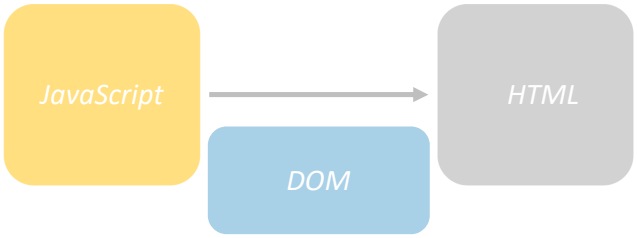
Document Object Model - DOM

Document Object Model (DOM)

- Um dos principais objetivos da utilização do *JavaScript* é a capacidade de manipular/alterar/controlar elementos HTML
  - DOM disponibiliza:
    - Métodos / Propriedades de forma a aceder a todo o documento HTML permitindo dessa forma a geração/alteração dinâmica de conteúdo HTML



Document Object Model

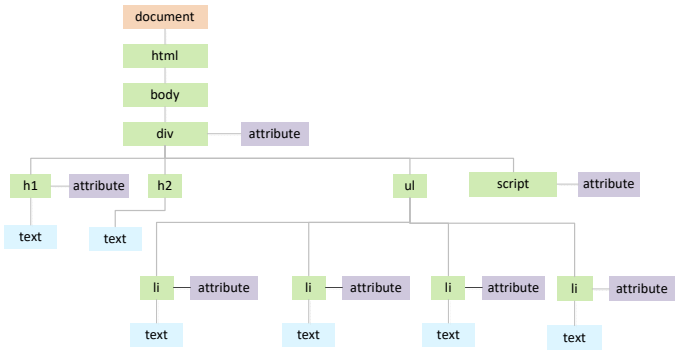
*The **HTML DOM** is a standard for how to get, change, add, or delete HTML elements.*

http://www.w3schools.com/js/js_htmldom.asp

Document Object Model (DOM)

Quando é feito o download de um documento HTML, este torna-se num ***document object***

- É o *root node* do document HTML e o *"owner"* de todos os outros nós:
  - element nodes; text nodes; attribute nodes; comment nodes
- O *document object* disponibiliza propriedades e métodos para aceder a todos os nós com base em JavaScript.



DOM tree

Document Object Model (DOM)

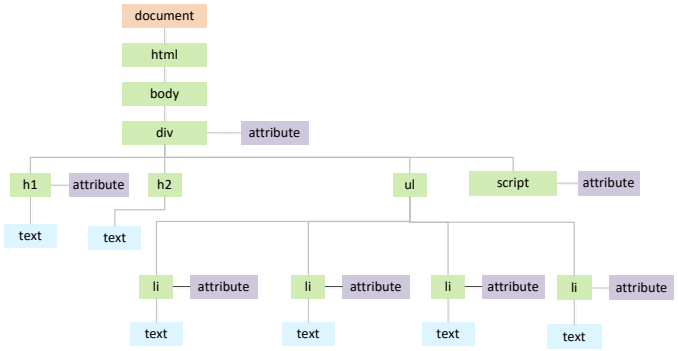
- HTML DOM considera tudo como nó (DOM Tree)
  - O documento (**root element**) é um ***document node***
  - Qualquer elemento HTML é um ***HTML node***
  - O texto contido nos elementos HTML é um ***text node***
  - Qualquer atributo HTML é um ***attribute node***

document

Element HTML

Text

Attribute



DOM tree

HTML DOM Reference

The references describe the properties and methods of each object, along with examples.

Attributes	Console	Document	Element	Events
Event Objects	Geolocation	History	HTMLCollection	Location
Navigator	Screen	Style	Window	Storage

Document Object

DOM

HTML DOM Reference

The references describe the properties and methods of each object, along with examples.

Attributes	Console	Document	Element	Events
Event Objects	Geolocation	History	HTMLCollection	Location
Navigator	Screen	Style	Window	Storage

DOM: Document Object

- *DOM Methods*

Método (*DOM Queries*)	Descrição
document.getElementById()	retorna o *element object* que tem o id com o valor especificado
document.querySelector()	retorna o primeiro *elemento object* referenciado pelo seletor CSS
document.querySelectorAll()	retorna o conjunto de elementos referenciados pelo seletor CSS
document.getElementsByTagName()	retorna conjunto de elementos *(HTML Collection)* cuja designação da tag corresponde ao nome especificado
document.getElementsByClassName()	retorna um conjunto de elementos *(HTML Collection)* que tem o atributo class com o nome especificado
…	…

http://www.w3schools.com/jsref/dom_obj_document.asp

- Sempre que necessário utilizar o mesmo elemento mais do que uma vez, o *element object* deve ser referenciado por uma variável:

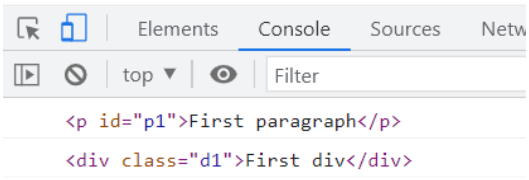
var identificaElemento = document.getElementById('one');

```
<p id="p1">First paragraph</p>
<div class="d1">First div</div>

<script>
    var a=document.getElementById('p1');
    var b=document.querySelector('.d1')

    console.log(a)
    console.log(b)
```

Métodos DOM que referenciam elementos HTML



Element Object

DOM

HTML DOM Reference

The references describe the properties and methods of each object, along with examples.

Attributes	Console	Document	Element	Events
Event Objects	Geolocation	History	HTMLCollection	Location
Navigator	Screen	Style	Window	Storage

DOM: Element Object

- O Element Object representa um elemento HTML

HTML Element Objects Reference

The references describe the properties and methods of each HTML object, along with examples.

a	abbr	address	area	article	aside	audio	b	base	bdo	blockquote	body	br	button	canvas	caption	cite	code	col	colgroup
datalist	dd	del	details	dfn	dialog	div	dl	dt	em	embed	fieldset	figcaption	figure	footer	form	head	header	h1 - h6	hr
html	i	iframe	img	ins	input button	input checkbox	input color	input date	input datetime	input datetime-local	input email	input file							
input hidden	input image	input month	input number	input password	input radio	input range	input reset	input search	input submit	input text									
input time	input url	input week	kbd	label	legend	li	link	map	mark	menu	menuitem	meta	meter	nav	object	ol	optgroup		
option	output	p	param	pre	progress	q	s	samp	script	section	select	small	source	span	strong	style	sub	summary	sup
table	tbody	td	tfoot	th	thead	tr	textarea	time	title	track	u	ul	var	video					

- possui propriedades/métodos
  - No entanto existem algumas propriedades/métodos adicionais que são especificas de alguns elementos
    - ex: propriedade **disabled** do Button object

https://www.w3schools.com/jsref/

DOM: Element Object

- Element Object
  - Representa um elemento HTML (ex: , <form>, <a>, …)
  - Tem métodos e propriedades associadas

Propriedades	Descrição
innerHTML	acesso/alteração do conteúdo HTML de um elemento
textContent	Acesso/alteração de texto
className	Retorna/altera o valor do atributo *class* de um elemento *: substitui todos os valores anteriormente definidos para o atributo class.
classList	Retorna todos os valores do atributo *class* de um elemento
id	acesso/alteração do atributo id de um elemento
…	…

métodos	Descrição
addEventListener()	adiciona um event handler ao elemento
getAttribute()	obtém o valor de um dado atributo do elemento

https://www.w3schools.com/jsref/dom_obj_all.asp

DOM: Element Object

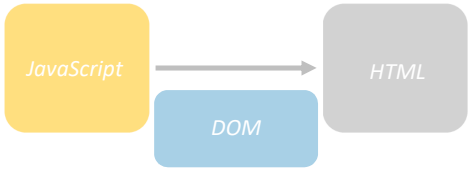
```
<p id="p1">First paragraph</p>

 <script>
   var a=document.getElementById('p1');
   a.textContent='CHANGED !'
```

Propriedade **textContent** permite alterar de forma dinâmica o conteúdo do element object referenciado pela variável **a**



Javascript altera/controla elementos HTML ?



Document Object

```
document.getElementById()
document.querySelector()
document.querySelectorAll()
document.getElementsByTagName()
document.getElementsByClassName()
...
```

Métodos que referenciam 1 ou vários elementos HTML

\+

Element Object

```
innerHTML
textContent
className
classList
id
...
```

Propriedades/métodos que permitem alterar conteúdos/características/estrutura HTML

HTML Collection Object

DOM

HTML DOM Reference

The references describe the properties and methods of each object, along with examples.

Attributes	Console	Document	Element	Events
Event Objects	Geolocation	History	HTMLCollection	Location
Navigator	Screen	Style	Window	Storage

DOM: HTML Collection

- HTMLCollection Object
  - Representa uma lista de elementos HTML
  - Permite indexação como se de um array se tratasse (**não é um array!**)
    - **Não permite** a utilização de métodos do objeto array.
    - Possui a propriedade length (muito útil!)
  - Método *item()*

```
var elements=document.getElementsbyClassName('vegetables');

if(elements.length>=1){
        var firstItem=elements.item(0);}
```

  - ***Array sintax***

```
var elements=document.getElementsbyClassName('vegetables');

if(elements.length>=1){
    var firstItem=elements[0];}
```

DOM: HTML Collection

```
<p id="p1" class="par">First paragraph</p>
<p id="p2" class="par">Second paragraph</p>

<script>

 var a=document.getElementsByClassName('par')
 a[1].textContent='CHANGED !'
```

a referencia um conjunto de elementos HTML (**HTML collection object**) através da sua class

Requer sempre indexação, índices iniciam em 0



Document Object

Seleção de Elementos (DOM nodes)

Seleção de Elementos

- *document.getElementById()*
  - a forma mais rápida e mais eficaz de aceder a um único elemento
  - retorna o nó cujo elemento tem o id especificado na *DOM Query*

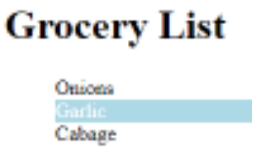
```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var veg = document.getElementById('two');
    veg.className='promo';
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```

1. A variável veg referencia o elemento HTML com id='two'
2. É atribuido o valor 'promo' ao atributo class da variável veg
3. É aplicada a formatação definida no código CSS

   (substitui o valor anterior, neste caso 'vegetables')



Seleção de Elementos

- *document.querySelector()*
  - Baseado em seletores CSS
  - Retorna o **primeiro elemento** que verifica o seletor CSS definido como argumento

```
<h1>Grocery List</h1>
<ul>
    <li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>


<script>
    var element=document.querySelector("#three");
    element.className='promo';
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```

1. A variável **element** referencia o elemento HTML com id='three'

   (querySelector() utiliza seletores CSS)
2. É atribuido o valor 'promo' ao atributo class da variável element
3. É aplicada a formatação definida no código CSS

   (substitui o valor anterior, neste caso 'vegetables')



Seleção de Elementos

- Métodos que retornam uma *HTML Collection*

Método (*DOM Queries*)	Descrição
document.getElementById()	retorna o *element object* que tem o id com o valor especificado
document.querySelector()	retorna o primeiro *elemento object* referenciado pelo seletor CSS
document.querySelectorAll()	retorna o conjunto de elementos *(HTML Collection)* referenciados pelo seletor CSS
document.getElementsByTagName()	retorna conjunto de elementos *(HTML Collection)* cuja designação da tag corresponde ao nome especificado
document.getElementsByClassName()	retorna um conjunto de elementos *(HTML Collection)* que tem o atributo class com o nome especificado
…	…

http://www.w3schools.com/jsref/dom_obj_document.asp

Seleção de Elementos

- *document.querySelectorAll()*
  - permite aceder aos *DOM nodes* com base num seletor CSS
  - Requer indexação mesmo que exista apenas um element ao qual se aplique o selector CSS

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
    <li id="four" class="fish">sardines</li>
    <li id="five" class="fish">codfish</li>
</ul>

<script>
    var elements,el;
    var elements = document.querySelectorAll('.fish');
    for (var i=0; i<elements.length; i++)
    {
        elements[i].className='promo';
    }
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```



1. A variável elements referencia todos os elementos HTML com class “fish”
2. Percorre elements e atribui a cada um dos elementos a class ‘promo’
3. É aplicada a formatação definida no código CSS

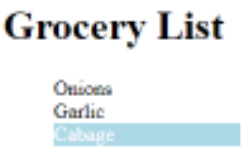
Seleção de Elementos

- *document.getElementsByTagName()*
  - retorna uma *HTML Collection com* todos os elementos que verificam o nome da *tag*

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var elements,el;
    var elements = document.getElementsByTagName('li');
    if (elements.length >2)
    {
        el=elements[2];
        el.className='promo';
    }
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```



1. A variável elements referencia **todos** os elementos HTML com a tag 'li'
2. Como a condição é verificada
3. É aplicada a formatação definida no código CSS ao elemento de indice [2]

Seleção de Elementos

- *document.getElementsByClassName()*
  - À semelhança do *getElementsByTagName()* retorna uma node list com os elementos com o valor do atributo *class* definido como argumento
  - Requer indexação

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var elements,el;
    var elements = document.getElementsByClassName('vegetables');
    if (elements.length >2)
    {
        el=elements[2];
        el.className='promo';
    }
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```



Seleção de Elementos

- Para todos os métodos que retornam um **HTML Collection Object**
  - Os elementos tem que ser indexados, mesmo que apenas exista **uma ocorrência** da tag name (posição [0]) **Importante!**

- Utilização de métodos de seleção de elementos diretamente numa condição
  - Caso o elemento exista é retornado o valor *true*
    - A presença de um *element object* é um ***truthy value***

```
if (document.getElementById('idValue'))
{
        /* processamento se o elemento existe */
}
else
{
       /* processamento se o elemento não foi encontrado*/
}
```


Seleção de Elementos

- *Form Elements Collection*
  - Forma alternativa de aceder aos elementos de um **formulário**
  - A *elements collection* retorna o conjunto de todos os elementos definidos num *form*
    - Os elementos estão ordenados de acordo com a ordem pela qual foram definidos no *form*
    - Indexados pela posição (inicio em 0) ou pelo valor do atributo id ou name.

```
function mostraElementos(){
    var el=document.getElementById('myForm')
    var txt=""

    for (var i=0; i<el.length;i++)
     txt += el[i].value + '<br>'

    document.getElementById('elementos').innerHTML=txt
  }
```

Seleção de Elementos

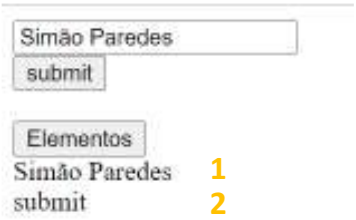
```
<form id="myForm">
  <input type="text" name="nome" placeholder="Nome completo"><br>
  <input type="submit" value="submit"><br>
 </form><br>

 <button onclick="mostraElementos()">Elementos</button>
 <div id="elementos"></div>


 <script>
  function mostraElementos(){
   var el=document.getElementById('myForm')
   var txt=""
   for (var i=0; i<el.length;i++)
     txt += el [i].value + '<br> '

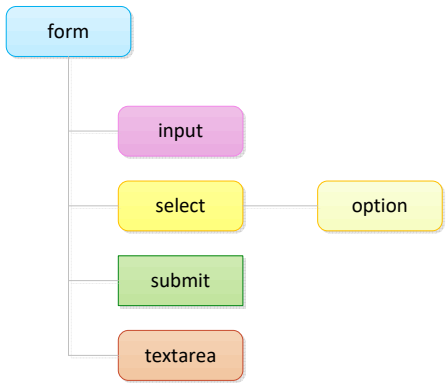
   document.getElementById('elementos').innerHTML=txt
  }
```

1
2



Document Object Model (DOM)

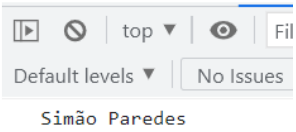
- Sintaxe alternativa para acesso aos elementos de um formulário



- Aceder a um campo do formulário:

document.getElementbyId('idForm').idElement

```
function mostraElementos(){
    var el=document.getElementById('myForm')
    console.log(el.d1.value)
    ...
```



Document Object

Criação de Elementos (DOM nodes)

Criação de Elementos

- Métodos

Método	Descrição
createElement()	cria um elemento HTML mas não adiciona à DOM tree
createTextNode()	cria um nó de texto mas não adiciona à DOM tree
createComment()	cria um *comment node* com o texto especificado
…	…

Criação de Elementos

- *createElement(); / createTextNode();*

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var elRef, elNew, elNewText;


    elNew=document.createElement("li");
    elNewText=document.createTextNode("Codfish");




    elNew.className="promo";
</script>
```

Criação de elementos

Element node

Text Node

Element Object

Alteração de Estrutura

Alteração de Estrutura

- Métodos

Método	Descrição
appendChild()	adiciona um nó à DOM tree
insertBefore()	insere um elemento antes de uma dada referência
replaceChild()	substitui um elemento por outro
removeChild()	remove um elemento

Alteração de Estrutura

- *appendChild()*

Adicionar à DOM Tree!

- Permite a integração/inclusão de um novo nó (elemento HTML / texto) no documento HTML que se pretende alterar
- Aceita como **único argumento** o nó que se **pretende adicionar**
- **O método deve ser chamado** no elemento que se pretende que seja o **pai** (estrutura HTML) do nó a inserir.

Alteração de Estrutura

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var elRef, elNew, elNewText;

    elRef=document.getElementsByTagName("ul")[0];
    elNew=document.createElement("li");
    elNewText=document.createTextNode("Codfish");

    elNew.appendChild(elNewText);
    elRef.appendChild(elNew);

    elNew.className="promo";
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```

Referência para posicionamento (elemento pai)

Adição dos elementos à DOM tree (elemento pai)



Alteração de Estrutura

- *insertBefore()*
  - Permite a inserção de um elemento antes de um dado elemento
  - Aceita dois argumentos: o primeiro é o nó a ser inserido e o segundo o nó (id) que determina o posicionamento

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

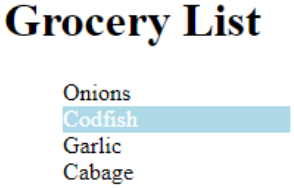
<script>
    var elRef, elNew, elNewText;

    elRef=document.getElementsByTagName("ul")[0];
    elNew=document.createElement("li");
    elNewText=document.createTextNode("Codfish");

    elNew.appendChild(elNewText);
    elNew.className="promo";

     elRef.insertBefore(elNew,document.getElementById("two"));
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```



Nó a ser inserido

Referência para posicionamento

Alteração de Estrutura

- *replaceChild ()*
  - Permite a substituição de um elemento por outro
  - Aceita dois argumentos: o primeiro é o nó a ser inserido e o segundo o nó (id) a ser substituído

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var elRef, elNew, elNewText;

    elRef=document.getElementsByTagName("ul")[0];
    elNew=document.createElement("li");
    elNewText=document.createTextNode("Codfish");

    elNew.appendChild(elNewText);
    elNew.className="promo";

    elRef.replaceChild(elNew,document.getElementById("two"));
</script>
```

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```

Grocery List

Onions
Codfish
Cabage

Nó a ser inserido

Nó a ser substituído

Alteração de Estrutura

- *removeChild ()*
  - apaga um nó que aceita como argumento
  - à semelhança do ***appendChild()*** o método ***removeChild()*** tem que ser **invocado no nó pai**

```
<h1>Grocery List</h1>
 <ul>
     <li id="one" class="vegetables">Onions</li>
     <li id="two" class="vegetables">Garlic</li>
     <li id="three" class="vegetables">Cabage</li>
 </ul>

 <script>
     var elRef, elRmv;

     elRef=document.getElementsByTagName("ul")[0];
     elRmv=document.getElementById("two");

     elRef.removeChild(elRmv);
 </script>
```

Elemento a ser removido

Grocery List

Onions
Cabage

Element Object

Alteração de Conteúdo (propriedades)

Alteração de Conteúdo

- *innerHTML*
  - a propriedade mais versátil uma vez que permite alterar/definir o conteúdo (texto/***markup***) de um dado elemento
  - alterar o *markup* de um elemento (ex: adicionar um link)

```
<h1>Grocery List</h1>
<ul>
    <li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var element=document.getElementById("three");
    element.innerHTML='<a href="https://www.fishforthought.co.uk">sardines</a>';
    element.className='promo';
</script>
```

Redefinição do conteúdo do elemento
(permite markup HTML)

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```

Grocery List

Onions
Garlic
sardines

Alteração de Conteúdo

- *textContent*
  - obtém/altera o conteúdo textual de um elemento e ignora todo o *markup* HTML contido por esse elemento

```
<h1>Grocery List</h1>
<ul>
    <li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>

<script>
    var element=document.getElementById("three");
    element.textContent='<a href="https://www.fishforthought.co.uk">sardines</a>';
    element.className='promo';
</script>
```

Redefinição do conteúdo textual do elemento
(**não permite markup HTML**)



Alteração de Conteúdo

document.write()

Vantagens:

Rápido e simples de mostrar o resultado da adição de conteúdo

Desvantagens:

Funciona corretamente quando é feito o download da página

Se utilizado depois efetua o **overwrite** do conteúdo anterior



element.innerHTML

Vantagens:

Permite a inserção de markup com menos código do que os métodos DOM

É mais rápido do que os métodos DOM

É a forma mais simples de remover todo o conteúdo de um elemento (string vazia).

Desvantagens:

Pode criar problemas com os *event handlers*, uma vez que origina a eliminação de todos os *child elements* os quais podem estar a ser utilizados para disparar um evento

Métodos DOM

Vantagens:

Permite aceder de forma precisa a todos os elementos da *DOM Tree.*

Não afeta os *event handlers*

Permite a adição incremental de elementos.

Desvantagens:

Caso se pretenda efetuar muitas alterações ao conteúdo é mais lento que *innerHTML*

Para atingir um determinado objetivo, necessita de mais código quando comparado com *o innerHTML*

Element Object

Alteração de Atributos

Alteração de Atributos

Método	Descrição
getAttribute()	obtém o valor do atributo de um elemento
hasAttribute()	verifica se o elemento possui um dado atributo
setAttribute()	define o valor de um atributo
removeAttribute()	remove um atributo de um elemento

Propriedades	Descrição
className	permite obter/estabelecer o valor do atributo class
classList	permite obter o valor do atributo class de um elemento (usada em conjunto com os métodos add() e remove(), toggle() para adicionar ou eliminar uma class)
style	permite definir/obter o valor de propriedades CSS
id	permite obter/estabelecer o valor do atributo id

http://www.w3schools.com/jsref/dom_obj_document.asp

Alteração de Atributos

- *getAttribute(); / setAttribute();*

```
<h1>Grocery List</h1>
<ul>
    <li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>
<p></p>

<script>
    var element, msg="";

    element=document.getElementById("three").getAttribute("class");
    msg+= "Old class attribute: " + element + "<br>";

    element=document.getElementById("three").setAttribute("class", "promo");
    element=document.getElementById("three").getAttribute("class");
    msg+= "New class attribute: " + element;

    document.getElementsByTagName("p")[0].innerHTML=msg;
</script>
```

Grocery List

Onions
Garlic
Cabage

Old class attribute: vegetables
New class attribute: promo

Acede ao valor do atributo class

Define um novo valor do atributo class

Acede ao novo valor do atributo class

- *setAttribute()* permite múltiplas alterações: alterar imagem (src); destino (href), …

Alteração de Atributos

- Propriedade ***style***
  - mais eficaz e expedito que os métodos anteriores para alteração do atributo *style*
    - retorna um *CSSStyleDeclaration* object o qual representa o atributo *style* de um elemento e que possui um conjunto alargado de propriedades

```
<h1>Grocery List</h1>
<ul>
    <li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
</ul>


<script>
    var element=document.querySelector("#three");
    element.style.backgroundColor='lightblue';
    element.style.color='white';
</script>
```

Grocery List

Onions
Garlic
Cabage

Propriedades do CSSStyleDeclaration object:

color
backgroundColor

Alteração de Atributos

- Propriedade ***style***
  - O nome das propriedades difere ligeiramente/pontualmente das propriedades CSS originais
    - Propriedades CSS com 2 palavras separadas por hífen são definidas em camelCase

Style Object Properties

Property	Description
alignContent	Sets or returns the alignment between the lines inside a flexible container when the items do not use all available space
alignItems	Sets or returns the alignment for items inside a flexible container
alignSelf	Sets or returns the alignment for selected items inside a flexible container
animation	A shorthand property for all the animation properties below, except the animationPlayState property
animationDelay	Sets or returns when the animation will start
animationDirection	Sets or returns whether or not the animation should play in reverse on alternate cycles
animationDuration	Sets or returns how many seconds or milliseconds an animation takes to complete one cycle

https://www.w3schools.com/jsref/dom_obj_style.asp

Alteração de Atributos

- Propriedade ***classList***
  - Propriedade muito importante para controlar a formatação de um elemento através de classes CSS previamente definidas.
  - Esta propriedade retorna um *DOMTokenList object* (objecto que contém uma lista de strings)

```
<h1>Grocery List</h1>
<ul>
    <li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="Veg1 Veg2 Veg3 Veg4">Cabage</li>
</ul>

<script>
    var element=document.querySelector("#three");
    var namesClass = element.classList;
    console.log(namesClass);
</script>
```

```
▼ DOMTokenList(4)
    0: "Veg1"
    1: "Veg2"
    2: "Veg3"
    3: "Veg4"
    length: 4
```

Alteração de Atributos

- Propriedade ***classList***

```
▼ DOMTokenList(4)
    0: "Veg1"
    1: "Veg2"
    2: "Veg3"
    3: "Veg4"
    length: 4
```

- Ao DOMTokenList object podem ser aplicados métodos muito importantes/úteis para controlar quais as classes aplicadas a um elemento:
  - add()
  - remove()
  - contains()

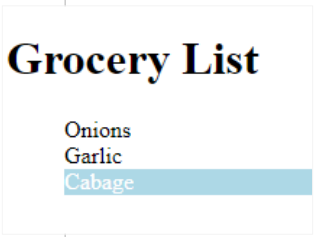
Alteração de Atributos

- Exemplo de controlo dinâmico da aplicação de seletores de class CSS
  - *contains(); add(); remove()*

```
<style>
    li{list-style-type: none}
    .promo{
        background-color:lightblue;
        color:white;
    }
</style>
```

```
<h1>Grocery List</h1>
<ul>
    <li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="Veg1 Veg2 Veg3 Veg4">Cabage</li>
</ul>

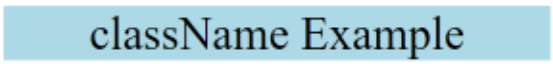
<script>
    var element=document.querySelector("#three");
    if (element.classList.contains('promo'))
        {element.classList.remove('promo');}
    else
        {element.classList.add('promo');}
</script>
```



Alteração de Atributos

- propriedade *className*
  - Exemplo:

```
<style>
    div{background-color: lightblue;
        width:30%;
        text-align: center;}
    .cl1{font-size: 2em;}
    .cl2{color:white}
</style>
</head>
<body>
    <div class='cl1' id='d1'>className Example</div>
```



  - className permite definir o valor da class, sendo que substituí sempre o valor anterior

```
<script>
    document.getElementById('d1').className='cl2';
```


Alteração de Atributos

- *className*
  - apesar de menos frequente é possível utilizar a propriedade *className* para adicionar valores ao atributo class sem substituir o anterior
    - A solução passa por utilizar uma atribuição composta com o novo valor (string) iniciar por espaço

```
<script>
    document.getElementById('d1').className +=' cl2';
```

Element Object

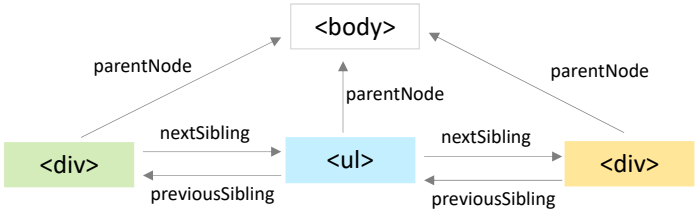
Seleção de Elementos (traversing properties)

Seleção de Elementos

- Traversing Properties
  - Têm por base a relação hierárquica (DOM Tree) dos elementos
    - previousSibling
    - nextSibling
    - firstChild
    - lastChild
    - parentNode
    - ...

```
<!DOCTYPE html>
<html>

<head>
    <title>My page</title>
</head>
<body>
    <div>Header</div>
    <ul>
        <li>List</li>
    </ul>
    <div>Footer</div>
</body>
</html>
```



- Traversing Properties

```
<h1>Grocery List</h1>
<ul><li id="one" class="vegetables">Onions</li>
    <li id="two" class="vegetables">Garlic</li>
    <li id="three" class="vegetables">Cabage</li>
    <li id="four" class="fish">sardines</li>
    <li id="five" class="fish">codfish</li>
</ul>
```

```
<script>
    var element, elBef, elNxt;
    element=document.getElementById('two');

    elBef=element.previousSibling;
    elBef.className='promo';

    elNxt=element.nextSibling;
    elNxt.className='promo';
</script>
```

Grocery List

- Onions
- Garlic
- Cabage
- sardines
- codfish

Window Object

HTML DOM Reference

The references describe the properties and methods of each object, along with examples.

Attributes	Console	Document	Element	Events
Event Objects	Geolocation	History	HTMLCollection	Location
Navigator	Screen	Style	Window	Storage

window Object

- Representa uma janela aberta no browser

Método	Descrição
window.open()	abre nova janela
window.close()	fecha a janela atual
window.moveTo()	movimenta a janela atual
window.resizeTo()	redimensiona a janela atual
window.print()	imprime a página atual
…	

- Métodos adicionais do window object (sem referência explicita ao objeto)

Método	Descrição
alert()	gera uma caixa de alerta
confirm()	gera uma caixa de confirmação
prompt()	permite a inserção de valores por parte do utilizador
setInterval()	chama repetidamente uma função com base num intervalo constante (ms)
setTimeout()	chama uma função após um determinado tempo (ms)
…	…

window Object

- Propriedades:

Propriedade	Descrição
window.innerHeight	altura da janela do browser em px
window.innerWidth	largura da janela do browser em px

window — *current browser window (top level object)*

- *history* — *pages in browser history*
- *location* — *URL of current page*
- *navigator* — *information about the browser*
- *screen* — *device's display information*

window Object

- Funções de Temporização
  - Particularmente interessantes :
    - Permitem controlar o tempo de chamada de uma outra função
      - Muito úteis por exemplo na implementação de *slideshows, …*
    - *setTimeout(nomeFunção, tempo(ms));*
      - chama a função após um período de tempo definido em ms

```
var randomNumber;
function alteraValor() {
    randomNumber = Math.random();
    document.getElementById('global').innerHTML = randomNumber;
}

setTimeout(alteraValor, 2000);
</script>
```

2 seg

0.7736800073180348

- *setInterval (nomeFunção, tempo(ms));*
  - chama uma função de forma repetida a intervalos regulares definidos em ms

```
<span id="global"></span> <br />

<script>
    var randomNumber;
    function alteraValor() {
        randomNumber = Math.random();
        document.getElementById('global').innerHTML = randomNumber;
    }
    setInterval(alteraValor, 2000);
</script>
```

0.06333285407163203 → *2 seg* → 0.9507259053643793 → *2 seg* → 0.3472738463897258

- *clearInterval()*
  - Elimina um *timer* que tenha sido estabelecido com *setInterval()*.
    - O *timer* deverá estar guardado numa variável que será o parâmetro do *clearInterval()*

```
var myVar = setInterval(function(){ myTimer() }, 1000);
...
function myStopFunction() {
    clearInterval(myVar);
}
```


Eventos